Loops

23 March 2026
09:13

loops are used to execute a block of code repeatedly until a specific condition is met or for each item in a collection

Types o LOOPs

1. **For Loop**
   It has 3 elements
   a. Counter Variable
   b. Range
   c. Increments or decrements

Example:

```
 for($i=0; $i -lt 10; $i++){
Write-host "hello-world" - Foregroundcolor "yellow"
 }
```

```
PS C:\Users\lenovo> for($i=0; $i -lt 10; $i++){
>> Write-host "hello-world" -foregroundcolor "red"
>> }
hello-world
hello-world
hello-world
hello-world
hello-world
hello-world
hello-world
hello-world
hello-world
hello-world
PS C:\Users\lenovo>
```

Here lt indicates - less then
Le : less than equal to

Example:

```
for($i=1;$i -le 10;$i++)
 {
 write-host "current-value of i is $i"
 write-host "hii Everyone" -foregroundcolor "blue"
}
```

```
PS C:\Users\lenovo> for($i=1;$i -le 10;$i++)
>> {
>> write-host "current-value of i is $i"
>> write-host "hii EVeryone" -foregroundcolor "blue"
>> }
current-value of i is 1
hii EVeryone
current-value of i is 2
hii EVeryone
current-value of i is 3
hii EVeryone
current-value of i is 4
hii EVeryone
current-value of i is 5
hii EVeryone
current-value of i is 6
hii EVeryone
current-value of i is 7
hii EVeryone
current-value of i is 8
hii EVeryone
current-value of i is 9
hii EVeryone
current-value of i is 10
hii EVeryone
```

Example of 2 table

```
for($i=1; $i -le 10;$i++){
write-host "2 x $i = $(2*$i)" -foregroundcolor "yellow"
}
```

```
PS C:\Users\lenovo> for($i=1; $i -le 10;$i++){
>> write-host "2 x $i = $(2*$i)" -foregroundcolor "yellow"
>> }
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
PS C:\Users\lenovo>
```

2. **While Loop**
   Another way of representing loops is through while loop
   Here the condition is checked first then the loop defined
   a. First define variable
   b. Apply condition
   c. Increment

   Example:

```
$a=1;
While($a -le 10)
{
Write-host "3 x $a = $(s*$a)"
```

$a++
}

```
PS C:\Users\lenovo> while($a -le 10)
>> {
>> write-host "3 x $a =$(3*$a)"
>> $a++
>> }
3 x  =0
3 x 1 =3
3 x 2 =6
3 x 3 =9
3 x 4 =12
3 x 5 =15
3 x 6 =18
3 x 7 =21
3 x 8 =24
3 x 9 =27
3 x 10 =30
PS C:\Users\lenovo>
```

Here the condition is checked first then the loop defined

3. **DO- While Loop**
   Here it is another way to implement loop
   Here at least once the body gets executed. If the condition is false the body of loop runs at least once.

a. Define variable
b. Increment
c. Define condition

$a=1
do{
write-host "3 x $a = $(3*$a)"
$a++
}while($a -le 10)

```
PS C:\Users\lenovo> $a=1
>> do{
>>  write-host "3 x $a = $(3*$a)"
>>  $a++
>> }while($a -le 10)
>>
>>
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
PS C:\Users\lenovo>
```

4. **Do-Until Loop**
   Until the condition is false the loop will run
   Here in this example the loop runs for infinity times.

$a=10
do{
write-host "3 x $a = $(3*$a)"
$a++
}while($a -lt 10)

Here in this example the a=10 and the condition is a should be less 10. so in this case the condition is not true so the loop runs

Example
$a=10;
Do{
Write-host "3 x $a = $(3*$a)"
$a--
}until($a -lt 10)

```
PS C:\Users\lenovo> $a=10;
>> Do{
>> Write-host "3 x $a = $(3*$a)"
>> $a--
>> }until($a -lt 10)
>>
3 x 10 = 30
PS C:\Users\lenovo>
```

In this case the loop runs for once where the conditions are met. Where the body is executed. As $a=10 is equal to 10(le) but not less then 10, the false conditions are executed.
If we make $a =11 which do not meet the condition but the loop runs as it is designed for running the false conditions

$a=11;
Do{
Write-host "3 x $a = $(3*$a)"
$a--
}until($a -lt 10)

```
PS C:\Users\lenovo>
>> $a=11;
>> Do{
>> Write-host "3 x $a = $(3*$a)"
>> $a--
>> }until($a -lt 10)
>>
>>
3 x 11 = 33
3 x 10 = 30
```

Twice the loop gets executed.

5. **For Each Loop**
   Used mostly with arrays
   Here every
   $b=@("tom", "mice", "jack")
   Foreach($item in $b)
   {
   Write-host "name is $item"
   }

```
PS C:\Users\lenovo> $b=@("tom", "mice", "jack")
>> Foreach($item in $b)
>> {
>> Write-host "name is $item"
>> }
>>
name is tom
name is mice
name is jack
PS C:\Users\lenovo>
```

   Incase of For Loop
   For ($i=0; $i -lt $b.length; $i++){
   Write-host "name is $($b[$i])"
   }

```
PS C:\Users\lenovo> For ($i=0; $i -lt $b.length; $i++){
>> Write-host "name is $($b[$i])"
>> }
>>
name is tom
name is mice
name is jack
PS C:\Users\lenovo>
```

BREAK AND CONTINUE

Break : come out of loop
Continue: skips one iteration, but loop runs

Example:
For($i=0; $i -lt $b.length ;$i++){
#break
If($i -eq 1){
Continue;
}
Write-host "Name is $($b[$i])"
}
So here the Mice statement is not executed because its index is 1. As per the condition we need to skip 1 iteration

```
PS C:\Users\lenovo> For($i=0; $i -lt $b.length ;$i++){
>> #break
>> If($i -eq 1){
>> Continue;
>> }
>> Write-host "Name is $($b[$i])"
>> }
>>
Name is tom
Name is jack
PS C:\Users\lenovo>
```

For($i=0; $i -lt $b.length ;$i++){
If($i -eq 1){
Break;
}
Write-host "Name is $($b[$i])"
}

```
PS C:\Users\lenovo> For($i=0; $i -lt $b.length ;$i++){
>> #break
>> If($i -eq 1){
>> break;
>> }
>> Write-host "Name is $($b[$i])"
>> }
>>
Name is tom
```